

Đánh giá Khoảng trống Nghiên cứu trong Kiến trúc Agentic GraphRAG: Tích hợp Hybrid RAG, Multi-Agent và Đồ thị Tri thức trong Hệ thống Tư vấn Tuyển sinh

1. Mở đầu và Bối cảnh Ứng dụng trong Giáo dục

Sự phát triển bùng nổ của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) đã định hình lại các hệ thống trí tuệ nhân tạo trong giáo dục, chuyển dịch từ các nền tảng Hỏi-Đáp (QA) tĩnh sang các hệ thống tương tác và suy luận phức tạp. Trong lĩnh vực tư vấn tuyển sinh đại học, người dùng thường đặt ra các truy vấn đòi hỏi độ chính xác tuyệt đối và khả năng suy luận đa bước (multi-hop reasoning), chẳng hạn như việc đối chiếu học phí, phân tích phổ điểm, hoặc theo dõi điều kiện tiên quyết của các ngành học chéo. Các hệ thống Retrieval-Augmented Generation (RAG) truyền thống hoạt động dựa trên việc so khớp vector tĩnh đang bộc lộ những điểm mù nghiêm trọng khi đối mặt với dữ liệu có tính cấu trúc và tính quan hệ cao của môi trường học thuật. Để giải quyết rào cản này, mô hình Agentic GraphRAG đã ra đời như một hệ sinh thái kiến trúc thể hệ mới, kết hợp khả năng duyệt đồ thị tri thức (Knowledge Graph) với các luồng công việc đa tác tử tự trị (Multi-Agent workflows)¹.

Các nghiên cứu thực nghiệm triển khai hệ thống Agentic GraphRAG trong môi trường đại học đã chứng minh những lợi ích vượt trội. Điển hình như hệ thống MARUAS (Multi-Agent RAG for University Admissions System) tại Việt Nam, kiến trúc đa tác tử kết hợp RAG đã giúp giảm tỷ lệ ảo giác (hallucination) từ 15% xuống chỉ còn 1.45%, đồng thời đạt độ chính xác trung bình 92% với độ trễ phản hồi dưới 4 giây trong các tác vụ tư vấn tuyển sinh thực tế³. Tương tự, khung nghiên cứu KA-RAG (Knowledge-Agent RAG) áp dụng đồ thị tri thức vào việc trả lời câu hỏi hướng khóa học cũng ghi nhận độ chính xác truy xuất đạt 91.4% và tính nhất quán ngữ nghĩa đạt 87.6%⁵. Mô hình Agentic GraphRAG đại diện cho sự giao thoa của hai nhánh tiến hóa: GraphRAG thay đổi cấu trúc dữ liệu nền tảng từ các đoạn văn bản phẳng (flat text chunks) sang mạng lưới các thực thể, trong khi Agentic RAG nâng cấp phương thức truy xuất từ chu kỳ đơn (single-shot) sang các vòng lặp tự phản biện, lập kế hoạch và hành động đa bước¹.

Tuy nhiên, việc tích hợp sâu cơ sở dữ liệu đồ thị như Neo4j, nền tảng điều phối đa tác tử như LangGraph, và cơ chế tìm kiếm lai (Hybrid Retrieval kết hợp Vector, BM25 và Graph) đang phát sinh nhiều rào cản kỹ thuật phức tạp. Phân tích dưới đây sẽ đi sâu vào 5 khoảng trống nghiên cứu (Research Gaps) cốt lõi của kiến trúc Agentic GraphRAG tính từ năm 2024 đến 2026, cung cấp cái nhìn học thuật toàn diện về những thách thức đe dọa trực tiếp đến tính khả thi khi triển khai hệ thống tư vấn tuyển sinh ở quy mô sản xuất (production-scale).

2. Khoảng trống Nghiên cứu 1: Sự Ảo giác

(Hallucination) và Quá tải Công cụ (Tool Overload) khi Sinh Truy vấn Dữ liệu Đặc thù

Khác với các hệ thống RAG thông thường xử lý ngôn ngữ tự nhiên, Agentic GraphRAG phụ thuộc tuyệt đối vào khả năng dịch truy vấn của người dùng sang ngôn ngữ truy vấn đồ thị (chủ yếu là Text2Cypher đối với hệ sinh thái Neo4j). Trong lĩnh vực giáo dục, nơi các con số về học phí, chỉ tiêu tuyển sinh, và điểm chuẩn mang tính đặc thù và nhạy cảm cao, việc trao cho một LLM Agent quyền tự do sử dụng công cụ tìm kiếm trên Đồ thị Tri thức (KG) rất dễ dẫn đến hiện tượng ảo giác cú pháp và ảo giác lược đồ (schema hallucination).

2.1 Ảo giác Cú pháp và Không gian Lược đồ (Schema Hallucination)

Nhiều báo cáo thực nghiệm chỉ ra rằng LLM thường gặp khó khăn lớn trong việc tạo ra các truy vấn Cypher có thể thực thi chính xác do sự bất đồng cấu trúc giữa ngôn ngữ tự nhiên và lược đồ của cơ sở dữ liệu. Khi đối mặt với các cấu trúc KG phức tạp, các phương pháp tạo truy vấn đơn bước (one-shot generation) thông qua in-context learning (ICL) thường thất bại trong việc ánh xạ các thực thể⁶. Ví dụ, một truy vấn về "học phí ngành Trí tuệ Nhân tạo năm 2026" có thể khiến tác tử tự động nội suy và bịa ra một thuộc tính quan hệ không tồn tại trong Neo4j như `[HAS_TUITION_FEE_2026]`, dẫn đến lỗi cú pháp hoặc trả về kết quả rỗng (empty result). Việc tác tử nhận được tập kết quả rỗng thường kích hoạt các nỗ lực tự sửa lỗi (self-correction) mù quáng, lãng phí tài nguyên token mà không giải quyết được nguyên nhân gốc rễ là sự bất đồng bộ về lược đồ⁸. Các hệ thống đánh giá mở rộng (như khung CyVer) cho thấy ngay cả các mô hình mã nguồn mở cỡ lớn cũng gặp giới hạn trong việc nắm bắt toàn vẹn ngữ nghĩa của lược đồ đồ thị phức tạp⁷.

2.2 Hiện tượng Quá tải Công cụ (Tool Overload) trong Điều phối Đa tác tử

Bên cạnh ảo giác lược đồ, các hệ thống đa tác tử còn đối mặt với hiện tượng "Tool Overload" (Quá tải công cụ). Việc cung cấp cho một tác tử một danh sách đồ sộ các công cụ khả dụng (từ công cụ tìm kiếm Vector, BM25, đến các hàm gọi Cypher đa dạng) thường làm suy giảm nghiêm trọng khả năng suy luận của nó. Cấu trúc mô tả của các công cụ (tool schema payload) chiếm dụng phần lớn không gian ngữ cảnh, khiến LLM bị "quá tải nhận thức", từ đó cố gắng ép buộc các truy vấn phức tạp đi qua các công cụ không chính xác hoặc ảo giác các tham số đầu vào để lấp đầy yêu cầu của công cụ⁹. Thay vì gọi một công cụ tính toán tổng hợp (aggregation tool) để tính điểm trung bình các môn xét tuyển, một tác tử bị quá tải có thể cố gắng lặp qua hàng trăm bản ghi sinh viên riêng lẻ thông qua công cụ phân trang (pagination), dẫn đến sự sụp đổ về mặt hiệu suất⁹.

2.3 Giải pháp Hiện hành và Khoảng trống Chờ giải quyết

Để đối phó với ảo giác Text2Cypher, một số kiến trúc gần đây như GraphRAFT đã đề xuất cơ chế "Giải mã Ràng buộc Dựa trên Cơ sở" (Grounded Constrained Decoding). Cơ chế này can thiệp trực tiếp vào bộ xử lý logit tại thời điểm sinh văn bản (test-time inference); nếu LLM dự đoán một token dẫn đến truy vấn Cypher không hợp lệ về mặt cú pháp hoặc vi phạm lược đồ, giá trị logit của token đó sẽ bị gán bằng âm vô cùng, buộc xác suất sinh ra nó bằng không¹⁰. Đồng thời, các hệ thống như CyVerACT đề xuất luồng làm việc đa tác tử cung cấp phản hồi

nhận thức về lược đồ (schema-aware feedback) khi câu lệnh Cypher bị lỗi, giúp cải thiện độ chính xác cú pháp lên đến 52.7% và độ chính xác khớp hoàn toàn (Exact Match) lên 13.5%⁶. Đối với vấn đề quá tải công cụ, hệ thống định tuyến ý định (Intent Router) được sử dụng để phân loại truy vấn theo các danh mục ngữ nghĩa khép kín (ví dụ: tìm kiếm thực thể, duyệt mạng lưới quan hệ, hoặc tính toán phân tích), từ đó giới hạn động (dynamically restrict) tập hợp các công cụ được tải vào ngữ cảnh của tác tử chính⁹.

Mặc dù giải mã ràng buộc (constrained decoding) đảm bảo tính hợp lệ của cú pháp Cypher, nó không hề đảm bảo tính chính xác về mặt ngữ nghĩa logic đối với ý định ban đầu của học sinh trong ngữ cảnh tư vấn tuyển sinh. Việc xác thực lược đồ liên tục trên một KG quy mô lớn ở thời gian thực tạo ra độ trễ tính toán đáng kể. Hơn nữa, việc tìm ra điểm cân bằng toán học giữa sự tự do khám phá (autonomous exploration) của tác tử và các ràng buộc thực thi (execution constraints) nghiêm ngặt đối với các dữ liệu đặc thù như học phí vẫn là một bài toán chưa có lời giải triệt để.

3. Khoảng trống Nghiên cứu 2: Bùng nổ Độ trễ (Latency) và Chi phí do Lạm dụng Vòng lặp Suy luận ReAct trong LangGraph

LangGraph đã trở thành khung tiêu chuẩn công nghiệp (industry-standard framework) cho việc xây dựng các hệ thống đa tác tử, nhờ khả năng mô hình hóa luồng công việc dưới dạng đồ thị trạng thái có hướng (stateful directed graphs). Kiến trúc này hỗ trợ luồng điều khiển linh hoạt, lưu trữ điểm kiểm tra (checkpointing), khả năng quay lui (retries) và can thiệp của con người (human-in-the-loop)¹³. Tuy nhiên, sự linh hoạt vô tiền khoáng hậu này đang bộc lộ một cái giá rất đắt về mặt hiệu suất khi xử lý các truy vấn giáo dục phức tạp.

3.1 Nút thắt Cổ chai của Vòng lặp Tự hồi quy (Autoregressive Bottleneck)

Trong mô hình ReAct (Reasoning and Acting) cốt lõi của Agentic RAG, tác tử liên tục xen kẽ giữa các bước suy nghĩ (thought), gọi công cụ (action), và nhận quan sát (observation) cho đến khi hoàn thành nhiệm vụ. Ở kiến trúc đa tác tử, mỗi lần một tác tử gửi thông điệp hoặc chuyển giao (handoff) nhiệm vụ cho một tác tử khác, toàn bộ ngữ cảnh (bao gồm bộ nhớ chuỗi hội thoại, các định nghĩa công cụ, lời nhắc hệ thống, và kết quả từ các bước truy xuất trước) phải được hệ thống LLM nạp và xử lý lại từ đầu (prefill phase)¹⁶. Sự phụ thuộc vào quá trình giải mã tự hồi quy khiến chi phí tính toán (đo lường bằng FLOPs) tăng tuyến tính với số lượng token, tạo ra nút thắt cổ chai về đọc/ghi bộ đệm KV-cache¹⁷.

Bảng dưới đây minh họa sự khác biệt rõ rệt về hiệu năng giữa các kiến trúc tổ chức tác tử (Agent Topologies) dựa trên các đo lường chuẩn đối sánh¹⁸:

Chỉ số Đo lường (Metric)	Tuần tự (Sequential)	Phân nhánh (Star/Supervisor)	Kết nối Toàn phần (Full Mesh)
Số lần gọi LLM	14.03	16.22 (+16%)	29.99 (+114%)

trung bình			
Thời lượng thực thi (s)	81.1	105.0 (+30%)	31.6 (-61%)
Token tiêu thụ mỗi luồng	25.5k	43.5k (+71%)	35.2k (+38%)
Độ trễ ngưỡng cao (p95)	8.52 s	16.20 s (+90%)	2.82 s (-67%)

Như dữ liệu cho thấy, mô hình Giám sát (Supervisor - dạng Star) mặc dù dễ kiểm soát nhưng lại tiêu thụ lượng token cao nhất và có độ trễ p95 cực đoan (16.20 giây). Lý do chính là tác tử giám sát liên tục phải đóng vai trò "phiên dịch" và phân tích ngữ cảnh từ các tác tử phụ sau mỗi bước, làm bùng nổ kích thước cửa sổ ngữ cảnh (context window)¹⁶.

3.2 Bùng nổ Trạng thái (State Space Explosion) và Tính Không Tất định

Một trong những cạm bẫy lớn nhất trong việc triển khai LangGraph cho các tác tử truy xuất là nguy cơ rơi vào các vòng lặp vô hạn (infinite iteration loops). Khi đối mặt với việc truy xuất thất bại (ví dụ: không tìm thấy mã ngành tương ứng), một tác tử được cấu hình để tự sửa lỗi có thể liên tục lặp lại các biến thể của truy vấn sai hàng chục lần nếu thiếu ngưỡng dừng cứng (iteration cap). Thực tế triển khai công nghiệp cho thấy độ trễ trung vị (p50) có thể duy trì ở mức 4-8 giây, nhưng độ trễ ở phần đuôi (p95 cost tail) có thể vọt lên 10-15 giây, đi kèm mức tiêu thụ token gấp 3 đến 10 lần so với RAG truyền thống¹⁹.

Hệ thống đánh giá hiệu năng XPerf nhấn mạnh rằng các ứng dụng AI theo hướng Agentic có sự biến động cực lớn về mẫu tải công việc. Đồ thị thực thi (execution graph) của hệ thống bị chi phối bởi các đầu ra giải mã không tất định (nondeterministic decoding) của LLM. Nghĩa là, hai học sinh hỏi cùng một câu hỏi về "quy chế tuyển sinh mới nhất" có thể kích hoạt hai quỹ đạo gọi công cụ (tool call trajectories) hoàn toàn khác nhau, khiến việc dự báo tài nguyên hệ thống (capacity planning) trở nên bất khả thi²⁰.

Khoảng trống nghiên cứu (Gap): Hiện tại, các cơ chế truyền tải thông tin giữa các nút (nodes) trong LangGraph vẫn chủ yếu dựa trên văn bản thuần túy (plain text), gây lãng phí nghiêm trọng tài nguyên KV-cache¹⁷. Các kỹ thuật chia sẻ bộ đệm (cache-to-cache) hoặc truyền trạng thái tiềm ẩn (latent state transmission) chưa được chuẩn hóa. Đặc biệt, việc xác định "Knee point" (điểm gãy) - điểm mà tại đó việc gia tăng số lượng vòng lặp Agentic không còn mang lại lợi ích về độ chính xác (Accuracy) nhưng lại làm suy thoái tuyến tính thông lượng (Throughput) - vẫn là một khoảng trống lớn cần nghiên cứu sâu hơn thông qua các mô hình tối ưu hóa toán học²¹.

4. Khoảng trống Nghiên cứu 3: Sự Thiếu Linh hoạt và Tối ưu hóa Kém trong Việc Dung hợp Vector, BM25 và Graph (Hybrid Retrieval)

Hệ thống tư vấn tuyến sinh là một môi trường chứa các truy vấn đa hình thái (mixed-intent queries). Các câu hỏi thiên về ngữ nghĩa như "Em có tính cách hướng ngoại thì hợp ngành nào?" yêu cầu sức mạnh của **Dense Vector Search** để so khớp khái niệm. Trái lại, các câu hỏi chứa mã định danh hoặc tên gọi chính xác như "Mã ngành của Kỹ thuật Phần mềm ĐHQG là gì?" lại bắt buộc sử dụng **Sparse Retrieval (BM25)**. Cuối cùng, các câu hỏi khai thác tính cấu trúc liên kết như "Các ngành thuộc Khoa Cơ khí có điều kiện tiên quyết là môn Toán cao cấp" chỉ có thể giải quyết bởi **Graph Retrieval**. Giải pháp lai (Hybrid RAG) về lý thuyết sẽ giải quyết được bài toán này, nhưng thực tiễn triển khai lại vấp phải rào cản nghiêm trọng về kiến trúc kết hợp (Fusion Architecture).

4.1 Khủng hoảng Bất đồng Thang điểm (Score Incompatibility)

Điểm yếu đầu tiên nằm ở sự không tương thích sâu sắc về phân phối toán học giữa các cơ chế tính điểm. BM25 (Best Match 25) là thuật toán từ vựng, tính điểm bằng cách nhân tần suất thuật ngữ (term frequency) với nghịch đảo tần suất tài liệu (inverse document frequency) và chuẩn hóa độ dài. Do đó, điểm BM25 là các số thực dương không bị giới hạn (unbounded positive integers), phụ thuộc vào độ hiếm của từ khóa trong kho lưu trữ²². Ngược lại, so khớp Dense Vector sử dụng độ tương đồng Cosine (Cosine Similarity) hoặc tích vô hướng, tạo ra các điểm số bị giới hạn chặt chẽ (thường từ $[-1, 1]$ hoặc $[0, 1]$).

Nếu hệ thống áp dụng phương pháp Tổ hợp Tuyến tính (Convex Combination / Weighted Alpha Fusion) đơn giản:

$$\text{score}_{\text{hybrid}} = \alpha \times \text{score}_{\text{dense}} + (1 - \alpha) \times \text{score}_{\text{sparse}}$$

thì hệ thống lập tức bị phá vỡ. Mà không có các phép chuẩn hóa (normalization) tinh vi như Z-score hay Min-Max, giá trị khổng lồ của BM25 sẽ vùi dập hoàn toàn điểm số của Vector, biến hệ thống Hybrid thành một cỗ máy tìm kiếm từ khóa trá hình²².

4.2 Nghịch lý của Reciprocal Rank Fusion (RRF)

Để lách qua bài toán bất đồng thang điểm, thuật toán **Reciprocal Rank Fusion (RRF)** đã nổi lên như một tiêu chuẩn tối thượng trong các $\mathbb{H}$ -line RAG (được mặc định trên Elasticsearch, Weaviate, v.v.). RRF hoàn toàn bỏ qua độ lớn của điểm số thô (raw score magnitudes), chỉ

đánh giá tài liệu dựa trên thứ hạng (rank) của chúng trong từng danh sách truy xuất R độc lập:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

(trong đó k là hằng số xếp hạng, thường thiết lập bằng 60^{26}).

Mặc dù RRF cung cấp sự ổn định xuất sắc vì không yêu cầu tinh chỉnh tham số, nó tạo ra một "khoảng mù" thông tin đáng lo ngại. Ví dụ, một tài liệu Vector xếp hạng nhất với độ tương đồng Cosine là 0.99 (gần như khớp tuyệt đối) sẽ nhận được điểm RRF y hệt như một tài liệu xếp hạng nhất nhưng có độ tương đồng chỉ là 0.51 (độ tin cậy thấp). RRF vứt bỏ hoàn toàn mức độ chênh lệch (magnitude) của khoảng cách tự tin này²⁸. Các bài báo cáo chuẩn đối sánh (benchmark) năm 2025-2026 trên tập dữ liệu WANDS cho thấy, khi hệ thống được tinh chỉnh đúng cách thông qua Relative Score Fusion (kết hợp chuẩn hóa điểm) hoặc sử dụng Learned Sparse (như thuật toán SPLADE), hiệu suất NDCG vượt trội hơn đáng kể so với RRF tĩnh²³.

4.3 Khó khăn khi Tích hợp Cấu trúc Liên kết (Graph Topology) vào Không gian Xếp hạng

Khoảng trống nghiên cứu này càng mở rộng khi tích hợp thêm GraphRAG. Vector và BM25 chia sẻ chung một định dạng đầu ra: một danh sách phẳng các đoạn văn bản (flat lists of text chunks). Trong khi đó, Graph Retrieval trả về các cấu trúc mạng lưới (neighborhood expansion, các đường dẫn đa bước, hoặc tóm tắt cụm cộng đồng - community summaries)³⁰.

Việc dung hợp kết quả có cấu trúc phi tuyến của Graph với danh sách tuyến tính của Vector/BM25 tạo ra một bài toán không tương thích về loại hình (type incompatibility). Một số kiến trúc hiện đại như VDGR-RAG thiết kế các luồng truy xuất này chạy song song, sau đó dùng công cụ LLM định tuyến (routing) phân tích riêng rẽ³¹. Các giải pháp tinh vi hơn liên quan đến việc nhúng đồ thị (Graph Embeddings) hoặc Graph Neural Networks (GNN) để đánh giá lại (rerank) các tài liệu, nhưng lại vấp phải chi phí tính toán cực lớn.

Khoảng trống nghiên cứu (Gap): Vẫn chưa có một thuật toán dung hợp (fusion framework) đa phương thức nào đủ linh hoạt để (1) không làm mất đi tín hiệu tự tin cục bộ của từ khóa, (2) bảo tồn được tính cấu trúc đa chiều của dữ liệu đồ thị, và (3) thực thi trong giới hạn độ trễ cho phép mà không bị đội chi phí từ việc sử dụng các mô hình xếp hạng chéo (Cross-encoder rerankers) liên tục.

5. Khoảng trống Nghiên cứu 4: Sự Suy thoái Tri thức Thời gian (Temporal Degradation) và Cập nhật Động trên Neo4j

Đặc thù lớn nhất của dữ liệu giáo dục và tuyển sinh là tính biến động liên tục theo thời gian (temporal dynamics). Chỉ tiêu xét tuyển, mức học phí, hay chính sách học bổng của ngành "Kỹ thuật Máy tính" năm 2024 có thể hoàn toàn khác biệt so với năm 2025 hoặc 2026. Hầu hết các kiến trúc GraphRAG hiện nay (kể cả phiên bản GraphRAG thương mại của Microsoft) hoạt động trên giả định rằng kho lưu trữ văn bản là tĩnh (static external corpora)³³.

5.1 Sự Đổ vỡ của Các Đồ thị Tri thức Phi Thời gian (Atemporal KGs)

Hệ thống GraphRAG gốc xây dựng đồ thị lấy thực thể làm trung tâm, trích xuất các mối quan hệ, nhóm chúng thành các "cộng đồng" (communities) thông qua thuật toán phân cụm cộng đồng, và tính toán trước các bản tóm tắt phân cấp (hierarchical summaries). Hạn chế chết người của kiến trúc này là bất kỳ một bản cập nhật thông tin nhỏ nào (chẳng hạn như thay đổi

học phí của một ngành học) cũng có thể làm vô hiệu hóa cấu trúc cộng đồng, kích hoạt quá trình tính toán lại (recomputation) toàn bộ cấu trúc đồ thị³³. Sự chậm trễ và tốn kém này đi ngược lại hoàn toàn với bản chất của Agent Memory, nơi mà tác tử cần liên tục học hỏi từ các luồng thông tin mới và dữ liệu tương tác theo thời gian thực. Hơn nữa, khi một hệ thống RAG không có nhận thức về chiều thời gian, một truy vấn như "Điểm chuẩn năm ngoái của ngành này là bao nhiêu?" sẽ khiến các công cụ tìm kiếm vector gặp khủng hoảng. Hệ thống rất dễ trộn lẫn dữ liệu (context mixing) của nhiều năm khác nhau hoặc trả về thông tin rác (stale context). Hiện tượng này được các nhà nghiên cứu gọi là "truy xuất các quy định đã chết" (citing dead regulations), dẫn đến việc tác tử đưa ra lời khuyên tuyến sinh sai lệch, gây hậu quả nghiêm trọng về độ tin cậy³⁵.

5.2 Mô hình Bi-Temporal và Rào cản Suy luận Chéo

Để khắc phục, các kiến trúc mới (từ cuối năm 2025 đến 2026) như động cơ Graphiti của nền tảng Zep, hay khung TG-RAG (Temporal Graph RAG) đã giới thiệu "Đồ thị tri thức nhận thức thời gian" (Temporally-aware Knowledge Graphs)³³. Đặc trưng cốt lõi của Graphiti là mô hình Bi-temporal (thời gian kép): hệ thống theo dõi cả thời điểm một sự kiện thực sự xảy ra (event time) và thời điểm nó được ghi nhận vào hệ thống (ingestion time). Mỗi cạnh (relationship) trong đồ thị Neo4j được bổ sung hai thuộc tính khoảng thời gian hiệu lực tường minh: t_{valid} và $t_{invalid}$ ³³.

Bảng sau so sánh phương pháp xử lý dữ liệu động của các hệ thống GraphRAG:

Cơ chế Hệ thống	Quản lý Dữ liệu Động (Dynamic Data Handling)	Xử lý Xung đột Thời gian (Temporal Conflicts)	Trạng thái Cộng đồng (Community State)
Microsoft GraphRAG	Tính toán lại theo lô (Batch recomputation)	Bị trộn lẫn, ghi đè (Overwritten)	Tóm tắt tĩnh (Static summaries)
TG-RAG	Đồ thị thời gian hai cấp (Bi-level temporal graph)	Trích xuất theo hạt thời gian (Time-nodes)	(Không áp dụng)
Zep / Graphiti	Cập nhật tăng dần thời gian thực (Real-time incremental)	Gắn cờ vô hiệu hóa thay vì xóa ($t_{invalid}$)	Duy trì độ chính xác lịch sử

Khi có dữ liệu mới mâu thuẫn với dữ liệu cũ (như quy chế mới bãi bỏ quy chế cũ), hệ thống

không xóa thông tin mà thông minh sử dụng siêu dữ liệu thời gian để làm mất hiệu lực (invalidate), bảo tồn tính toàn vẹn lịch sử mà không cần tái cấu trúc toàn hệ thống³³.

Khoảng trống nghiên cứu (Gap): Dù đã có kiến trúc lưu trữ Bi-temporal như Zep/Graphiti, khoảng trống nghiên cứu chuyển sang bài toán suy luận đa mốc thời gian (cross-temporal reasoning) của LLM Agents. Việc buộc một tác tử tự động lập kế hoạch đa bước và sinh ra các lệnh Cypher phức tạp để theo dõi sự biến đổi của một thuộc tính dọc theo dòng thời gian (long-horizon temporal reasoning) vẫn gây ra tỷ lệ lỗi cực cao. Khung đánh giá LongMemEval chỉ ra rằng, ngay cả những tác tử được trang bị LLM tinh vi nhất, khả năng giữ vững logic khi thực hiện các phép chiếu lịch sử (historical state reconstructions) vẫn là một vấn đề nhức nhối³⁸.

6. Khoảng trống Nghiên cứu 5: Sự Khuyết thiếu Các Khung Đánh giá (Evaluation Frameworks) Chuyên dụng cho Hệ thống Đa tác tử

Sự chuyển dịch kiến trúc từ RAG đơn tác tử (single-agent) sang Agentic GraphRAG (multi-agent) đã làm mất đi tính hiệu lực của các chuẩn mực đo lường truyền thống. Việc sử dụng các thang đo so khớp n-gram như BLEU, ROUGE, hoặc các chỉ số đánh giá LLM-as-a-judge truyền thống (như Answer Relevance hay Faithfulness trong RAGAS) chỉ kiểm tra đầu ra cuối cùng (end-result metrics). Chúng hoàn toàn "mù" trước luồng tương tác nội bộ, cách các tác tử phối hợp, cũng như quá trình duyệt và trích xuất đồ thị bên dưới.

6.1 Sự Phức tạp của Việc Đánh giá Dựa trên Thực thi (Execution-Grounded Evaluation)

Trong một vòng lặp công việc LangGraph phức tạp, một tác tử có thể liên tục phân rã nhiệm vụ sai, sử dụng công cụ Cypher lỗi hàng chục lần, dựa dẫm vào các nỗ lực tự sửa lỗi, và tiêu tốn một lượng tài nguyên khổng lồ trước khi tình cờ trích xuất được câu trả lời đúng⁴¹. Nếu chỉ quan sát kết quả cuối cùng, hệ thống được đánh giá là hoàn hảo. Tuy nhiên, ở góc độ kỹ thuật (system-level performance), tác tử đó đã tạo ra chi phí tính toán (token consumption) không bền vững và độ trễ không thể chấp nhận trong môi trường tư vấn trực tiếp (real-time counseling). Từ năm 2025, các khung đánh giá chuyên sâu cho hệ thống đa tác tử như MASEval, REALM-Bench, và "Mind the Query" đã được giới thiệu để lấp đầy khoảng trống này⁴².

- **MASEval** mang đến công cụ đánh giá độc lập với framework (framework-agnostic), đo lường tác động của các quyết định triển khai cấu trúc liên kết đa tác tử (topology) thay vì chỉ tập trung vào sức mạnh nội tại của LLM⁴⁴.
- **REALM-Bench** cung cấp bài kiểm tra về khả năng phối hợp của đa tác tử trước các gián đoạn môi trường (dynamic disruptions), đòi hỏi hệ thống phải liên tục thích ứng và lập kế hoạch lại (replanning)⁴².
- Đối với tác vụ Text2Cypher trên Neo4j, framework **"Mind the Query"** đề xuất phương pháp đánh giá hoàn toàn dựa trên khả năng thực thi thực tế (execution-grounded) thay vì chỉ so sánh chuỗi truy vấn (string matching)⁴³. Nó đo lường tỷ lệ các câu lệnh Cypher trả về kết quả rỗng (empty predictions) và tỷ lệ vi phạm ngữ nghĩa đồ thị.

6.2 Nghịch lý Đánh giá (The Evaluator Paradox) và Độ lệch Vị trí

Bên cạnh đó, ngành công nghiệp đang ghi nhận hiện tượng "The Evaluator Paradox" (Nghịch lý Đánh giá) đối với các hệ thống Agentic GraphRAG¹⁹. Các công cụ sử dụng LLM làm giám khảo (LLM-as-a-judge) hiện nay gặp phải "độ lệch vị trí" (position bias) nghiêm trọng. Trong các cuộc đối đầu đánh giá, LLM giám khảo thường đánh giá cao các câu trả lời mang tính tổng quát từ GraphRAG (thể hiện tính Đa dạng - Diversity) hơn là các thông tin vi mô, chính xác từ Vector RAG (thể hiện tính Toàn diện - Comprehensiveness)⁴⁵. Đặc biệt trong dữ liệu tuyển sinh đại học (ví dụ: một quy tắc xét tuyển kết hợp điểm thi, vùng miền, và chứng chỉ ngoại ngữ), các Agentic Evaluator thường không sở hữu đủ năng lực toán học mệnh đề hoặc logic quan hệ để chứng minh hệ thống con của mình đã lập luận đúng hay sai².

Khoảng trống nghiên cứu (Gap): Việc thiếu hụt một hệ thống theo dõi từ xa (telemetry) có thể ghi nhận tường tận sự suy luận của Agent qua từng bước trên Neo4j (như số lần thử lại, nguyên nhân thất bại của từng lần, và sự phân bổ chi phí token trên mỗi trường thông tin - amortized cost) vẫn là một rào cản lớn²¹. Thêm vào đó, hầu như chưa có một bộ cơ sở dữ liệu chuẩn đối sánh (benchmark dataset) chuyên sâu nào bằng tiếng Việt cho mảng đồ thị giáo dục, khiến quá trình đánh giá và tối ưu hóa các mô hình Agentic GraphRAG nội địa gặp rất nhiều khó khăn.

7. Kết luận

Triển vọng đưa các hệ thống AI ứng dụng trong tư vấn giáo dục vượt khỏi kỷ nguyên Hỏi-Đáp (QA) tĩnh dựa trên tài liệu phẳng (PDFs) đang phụ thuộc rất lớn vào kiến trúc **Agentic GraphRAG**. Nền tảng này hứa hẹn hợp nhất năng lực lập kế hoạch tự trị của Đa tác tử (Multi-Agent), tính minh bạch cấu trúc của Đồ thị Tri thức (Neo4j), và độ phủ tìm kiếm rộng lớn của các bộ truy xuất Hybrid. Tuy nhiên, sự kết hợp đan chéo này đã làm nảy sinh 5 khoảng trống nghiên cứu cốt lõi từ 2024 - 2026:

1. Rủi ro ảo giác không gian lược đồ (schema hallucination) và hiện tượng quá tải công cụ (tool overload) khi dịch ngôn ngữ tự nhiên sang truy vấn Cypher.
2. Nút thắt cổ chai về độ trễ và chi phí do sự tiêu tốn vòng lặp bộ đệm KV-cache trong các chu trình suy luận ReAct của LangGraph.
3. Sự bất đồng cấu trúc toán học trong việc dung hợp đa phương thức giữa thuật toán phân bổ rời rạc (BM25), không gian liên tục (Dense Vector), và mạng lưới liên kết (Graph Topology).
4. Tính cứng nhắc của các đồ thị tri thức tĩnh, cản trở việc theo dõi động lực thay đổi số liệu tuyển sinh theo thời gian thực mà không cần tái cấu trúc.
5. Sự khuyết thiếu các bộ khung đo lường (evaluation frameworks) có khả năng định lượng sự phối hợp đa tác tử nội bộ và hiệu suất thực thi Cypher thực tế.

Trong tương lai, các nỗ lực học thuật và kỹ thuật cần ưu tiên phát triển mô hình giải mã ràng buộc (constrained decoding) tối ưu để kiểm soát ảo giác; phát minh các phương pháp truyền trạng thái tiềm ẩn giữa các tác tử thay vì văn bản thuần túy; nghiên cứu các chuẩn mực dung hợp điểm số mới mẻ thay thế cho RRF tĩnh; và tập trung xây dựng cơ sở hạ tầng đồ thị thời gian kép (Bi-temporal graphs) như Graphiti nhằm bảo vệ tính toàn vẹn của chuỗi dữ liệu lịch sử. Việc chinh phục các rào cản này không chỉ đóng góp to lớn vào công nghệ xử lý ngôn ngữ tự

nhien, mà còn định hình chuẩn mực mới cho các hệ thống giáo dục số lấy sự chính xác và tin cậy làm cốt lõi.

Nguồn trích dẫn

1. (PDF) A Survey of Agentic GraphRAG: From Retrieval-Augmented, https://www.researchgate.net/publication/404635891_A_Survey_of_Agentic_GraphRAG_From_Retrieval-Augmented_Generation_to_Graph-Native_Agents
2. A Flexible Modular Framework for Retrieval-Augmented Generation, https://www.researchgate.net/publication/394271292_FRAG_A_Flexible_Modular_Framework_for_Retrieval-Augmented_Generation_based_on_Knowledge_Graphs
3. An Empirical Study of Multi-Agent RAG for Real-World University, <https://arxiv.org/html/2507.11272v1>
4. [2507.11272] An Empirical Study of Multi-Agent RAG for Real-World, <https://arxiv.org/abs/2507.11272>
5. KA-RAG: Integrating Knowledge Graphs and Agentic Retrieval, <https://www.mdpi.com/2076-3417/15/23/12547>
6. (PDF) CyVerACT: An Agentic Cypher Translation Workflow over, https://www.researchgate.net/publication/404561750_CyVerACT_An_Agentic_Cypher_Translation_Workflow_over_Knowledge_Graphs
7. (PDF) Decoding the Mystery: How Can LLMs Turn Text Into Cypher, https://www.researchgate.net/publication/391545599_Decoding_the_Mystery_How_Can_LLMs_Turn_Text_Into_Cypher_in_Complex_Knowledge_Graphs
8. Extending Confidence-Based Text2Cypher with Grammar ... - arXiv, <https://arxiv.org/pdf/2605.10318>
9. Agentic GraphRAG: Navigating Unstructured Financial Data ... - arXiv, <https://arxiv.org/html/2605.18770v1>
10. GraphRAFT: Retrieval Augmented Fine-Tuning for Knowledge, <https://www.alphaxiv.org/abs/2504.05478v1>
11. GraphRAFT: 그래프 데이터베이스 기반 지식 그래프를 위한 검색 증강, <https://www.alphaxiv.org/ko/abs/2504.05478v2>
12. Improving LLMs on Cypher generation via LLM-supervised, https://www.researchgate.net/publication/392506031_Auto-Cypher_Improving_LLMs_on_Cypher_generation_via_LLM-supervised_generation-verification_framework
13. (PDF) Implementing Multi-Agent Systems Using LangGraph, https://www.researchgate.net/publication/389497305_Implementing_Multi-Agent_Systems_Using_LangGraph_A_Comprehensive_Study
14. LangGraph: Multi-Agent Workflows - LangChain, <https://www.langchain.com/blog/langgraph-multi-agent-workflows>
15. LLM-Based Multi-Agent Orchestration: A Survey of Frameworks, <https://www.preprints.org/manuscript/202604.2147>
16. Benchmarking Multi-Agent Architectures - LangChain, <https://www.langchain.com/blog/benchmarking-multi-agent-architectures>

17. A Unified Framework for Latent Communication in LLM-based Multi, <https://arxiv.org/html/2606.05711v3>
18. Towards Traffic Modelling of Multi-Agent Systems: The Role ... - arXiv, <https://arxiv.org/html/2608.20494v1>
19. Agentic RAG: The 2026 Production Guide - MarsDevs, <https://www.marsdevs.com/guides/agentic-rag-2026-guide>
20. Benchmarking LLM Serving Systems for Agentic AI Workloads with, <https://arxiv.org/html/2608.20370v1>
21. Benchmarking Multi-Agent LLM Architectures for Financial ... - arXiv, <https://arxiv.org/html/2603.22651v1>
22. Hybrid Search: BM25, Vector & Reranking 2026 - Digital Applied, <https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026>
23. Hybrid RAG: Dense and Sparse Retrieval for Better AI Answers - Atlan, <https://atlan.com/know/hybrid-rag/>
24. Section-Weighted Hybrid Approach for Legal Case Retrieval - arXiv, <https://arxiv.org/pdf/2606.03138>
25. Score Fusion: RRF vs Relative Score Fusion - The AI Database Blog, <https://theaidatabaseblog.com/learn/score-fusion/>
26. Reciprocal rank fusion | Elasticsearch Reference, <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion>
27. Hybrid Retrieval Systems - 2026 Modern AI Search & RAG Roadmap, <https://nemorize.com/roadmaps/2026-modern-ai-search-rag-roadmap/lessons/hybrid-retrieval-systems>
28. Hybrid Search in Production: Why BM25 Still Wins on the Queries, <https://tianpan.co/blog/2026-04-12-hybrid-search-production-bm25-dense-embeddings>
29. Hybrid Search for RAG: Combining BM25 and Dense Vector Search, <https://denser.ai/blog/hybrid-search-for-rag/>
30. A Robust Multi-Hop GraphRAG Retrieval Framework - arXiv, <https://arxiv.org/html/2603.14828v1>
31. VDGR-RAG: Vectors, Directories, Graphs, and Reflection Are ... - arXiv, <https://arxiv.org/html/2608.07994v2>
32. VDGR-RAG: Vectors, Directories, Graphs, and Reflection Are ... - arXiv, <https://arxiv.org/html/2608.07994v1>
33. Graphiti: Knowledge graph memory for an agentic world - Neo4j, <https://neo4j.com/blog/developer/graphiti-knowledge-graph-memory/>
34. GraphRAG vs TemporalRAG: Which One Should You Actually Use, <https://www.youtube.com/watch?v=L9NkyJvxvY>
35. Graph-Powered Temporal Validity: Preventing RAG Systems ... - Neo4j, <https://neo4j.com/nodes/agenda/graph-powered-temporal-validity-preventing-rag-systems-from-citing-dead-regulations/>
36. Knowledge Graphs as Memory: Why Your AI Agent Needs to Think, <https://www.octoco.ai/blog/knowledge-graphs-as-memory>

37. IA-RAG: Interval-Algebra–Driven Temporal Reasoning for Dynamic,
<https://arxiv.org/html/2606.06044v1>
38. Zep: A Temporal Knowledge Graph Architecture for Agent Memory,
<https://arxiv.org/abs/2501.13956>
39. Open-Source Agent Memory: Mem0 vs Letta vs Zep - Digital Applied,
<https://www.digitalapplied.com/blog/open-source-agent-memory-mem0-letta-zep-compared>
40. State of AI Agent Memory 2026: Benchmarks & Trends Report - Mem0,
<https://mem0.ai/blog/state-of-ai-agent-memory-2026>
41. State of Agent Engineering - LangChain,
<https://www.langchain.com/state-of-agent-engineering>
42. REALM-Bench: A Benchmark for Evaluating Multi-Agent Systems on,
<https://arxiv.org/pdf/2502.18836>
43. PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text,
<https://arxiv.org/html/2606.08481v1>
44. MASEval: Extending Multi-Agent Evaluation from Models to Systems,
<https://arxiv.org/html/2603.08835v1>
45. RAG vs. GraphRAG: A Systematic Evaluation and Key Insights,
<https://www.alphaxiv.org/abs/2502.11371>
46. Multi-Agent Evaluation Suite for Testing, Reliability, and Observability,
<https://arxiv.org/html/2601.00481v1>